Filter-Induced and Learned Deadlocks in Safety-Filtered Reinforcement Learning for Mobile Robot Navigation

Pouya Bathaei Pourmand

Draft manuscript, not submitted or peer reviewed
8 November 2026

Abstract

A control-barrier-function (CBF) safety filter can make a learned navigation policy collision-free, but the robot may then stall in front of an obstacle and time out. We analyse such deadlocks for a unicycle robot whose speed is limited by a one-step discrete-time CBF filter. Two elementary properties of the filter frame the analysis. Because the next position depends only on the speed, the one-step safety guarantee does not depend on the commanded turn rate, so any modification of the turn rate preserves it. Near the keep-out boundary, the set of headings that permit motion shrinks to the half-plane that points away from the obstacle. We then classify the timeouts of ten proximal policy optimisation (PPO) agents and two classical controllers on 300 seeded test scenarios. In every scenario, a geometric oracle confirms that a path keeping the safety margin exists. Two mechanisms dominate, and they depend on how the agent was trained. Agents trained without the filter and run with it are *blocked by the filter* at the boundary (141 of 175 expert timeouts). Agents trained with the filter mostly *stop by themselves* before the boundary (122 of 195), so the filter is not the immediate cause. In paired comparisons on the expert stage, training with the filter lowered the filter-blocked rate by 14.8 percentage points (95 % interval [5.6, 24.4]) and raised the self-stop rate by 17.6 points [8.2, 27.0]. Sampling actions instead of using the policy mean removed most self-stops but no filter-blocked stalls. Both training contrasts replicated in a pre-registered evaluation on 500 previously unused scenarios. In the same evaluation, a test-time heading modification that cannot weaken the safety guarantee lowered the filter-blocked rate of agents trained without the filter by 7.4 points [4.8, 10.2] and raised their success by 7.2 points [5.0, 9.7], with zero collisions. It left self-stops unchanged. Self-stops did not disappear with longer training: they were more frequent at the final training checkpoints than at the selected ones. In three further pre-registered training experiments (five seeds and 300k steps per arm), self-stops were removed partly by the published correction penalty and almost entirely by a Beta policy or by a reward without the risk penalty. In each case, filter-blocked stalls took their place. Total deadlock changed by at most about 5 points, and only the reward change raised success (by 6.2 points [1.2, 11.2]). A classical local planner shows the same exchange in reverse. It is filter-blocked whenever its own margin is smaller than the filter's; with a larger margin, it stalls in its own local minima instead. On a corrected scenario generator that honours the requested goal distances, all findings except one null result replicated, and expert success roughly halved. All results are from simulation with exact state and static circular obstacles, and they do not establish real-world safety.

Keywords: safe reinforcement learning; control barrier functions; safety filter; deadlock; mobile robot navigation; liveness.

1 Introduction

Safety filters are a common way to make learned controllers usable in safety-critical settings [1, 2]. The learned policy proposes an action, and a filter, often a control barrier function (CBF) quadratic program [3, 4], replaces it with the closest action that keeps the system in a safe set. In robot navigation, the safe set excludes a margin around every obstacle, and the filter guarantees under its modelling assumptions that the robot never enters that margin.

The guarantee says nothing about reaching the goal. For model-based nominal controllers it is known that CBF filters can create undesired stable equilibria on the boundary of the safe set [5, 6], and multi-robot CBF controllers can deadlock [7–9]. When the nominal controller is a reinforcement-learning (RL) policy, the situation is less clear. The policy may have been trained with the filter in the loop [10–12], it reacts to observations rather than to the true state, and its behaviour near obstacles is learned rather than designed. Recent work shows that RL under a perfect, least-restrictive filter loses no optimality [13], and that adding a penalty on the filter's correction improves the performance of filter-trained agents in trajectory tracking [10].

That safety mechanisms can impede task progress has been observed before. Yang et al. note that a policy unaware of the CBF can behave conservatively [11]. Thumm et al. note that fail-safe interventions can cause abrupt braking or aggressive steering [14]. Banerjee et al. note that shields with fixed backup policies often hinder task progress [15]. In crowd navigation, a planner may decide that all forward paths are unsafe and freeze the robot, although a feasible path typically exists [16]. Our targeted search did not find a study that, for a single robot driven by a learned policy behind a CBF filter, (i) separates stalls caused by the filter from stops commanded by the policy, (ii) checks every failure against a geometric feasibility oracle, and (iii) relates the dominant mechanism to how the policy was trained. This is the gap we address. The search was targeted, not systematic, so the claim is limited to the work we examined.

This paper studies that question in a deliberately simple and fully reproducible setting. The robot is a unicycle, the obstacles are static circles, and the filter is a one-step discrete-time CBF on the speed [17]. The setting is simple enough that the filter's behaviour can be analysed exactly, and that every timeout can be checked against a geometric feasibility oracle.

Research questions.

RQ1 Which properties of a one-step discrete-time CBF speed filter determine where and how a filtered robot can stall?

RQ2 By which mechanisms do learned policies deadlock behind such a filter, and does training with the filter change the mechanism?

RQ3 Which simple remedies (stochastic deployment, the published correction penalty, heading-level modification) remove which mechanism?

Contributions.

1. Elementary properties of the one-step speed filter for the unicycle (Section 4). They follow directly from the forward-Euler model and are not new theory. We state them because they determine which remedies can work and give testable predictions. One-step safety is independent of the commanded turn rate (Lemma 2), so heading-level remedies keep the guarantee. The set of headings that permit motion collapses as the barrier approaches zero (Proposition 1), which gives a stall bound (Corollary 1). At steps where the policy commands

a near-zero speed, no test-time speed limit can make the robot move (Lemma 3). This says nothing about earlier steps or about training.

2. A diagnostic procedure for deadlocks in this filtered navigation setting. It combines a geometric feasibility oracle, a stall taxonomy (filter-blocked vs. self-stop) and uncertainty estimates over both training seeds and test scenarios (Section 5). The oracle and the class thresholds are specific to static circular obstacles and a unicycle, but the procedure can be adapted to other settings.

3. Empirical evidence from ten PPO agents and two classical controllers (Section 6). The two stall mechanisms exist, and they depend on the training setup. They respond differently to stochastic deployment and to a heading modification. They replicate on previously unused scenarios and on a corrected scenario generator. The filter-blocked stalls match the geometry predicted by Proposition 1, and self-stops are not an artefact of early checkpoint selection. Pre-registered training experiments show that the published correction penalty, a Beta policy and a reward change each remove self-stops, wholly or in part, but that filter-blocked stalls take their place. A classical local planner shows the same exchange.

We do not claim a new safety filter or a new training algorithm. The safety proof of the filter is standard [17] and is restated only for completeness. The correction penalty is the established method of Pizarro Bejarano et al. [10], and similar penalties on blocked or corrected actions appear in earlier work [12, 18, 19]. We implement only its correction-penalty term, $\alpha\|u - u^{\text{exec}}\|^2$ with $\alpha = 1$. Here u is the policy's sample and u^{exec} is the executed action after the action bounds and our one-step filter. Our filter replaces their predictive safety filter, and we do not use their safe resets. We use this term as a baseline. What is new is the question it is used to answer: does it remove either stall mechanism in navigation, and which one (RQ3)?

2 Related work

Safety filters and CBFs. CBF quadratic programs [3, 4] and their discrete-time form [17, 20] are the standard tools for enforcing state constraints around arbitrary nominal controllers. High-order CBFs handle constraints with higher relative degree [21]. Predictive and reachability-based filters offer less conservative alternatives at higher computational cost [1, 22]. Robust, measurement-robust and stochastic variants address model and state uncertainty [23–26].

Undesired equilibria and deadlock. Reis et al. [5] showed that CBF-QPs can introduce asymptotically stable equilibria on the safe-set boundary, and Tan and Dimarogonas [6] characterised them in general. In multi-robot navigation, deadlock was analysed as a force equilibrium on the boundary [7, 27] and resolved through instability of the deadlock set [8] or with discrete-time CBFs [9]. All of these analyses assume a model-based nominal controller. Potential fields have the analogous problem of local minima [28]. In crowd navigation, the "freezing robot problem" describes a planner that decides all forward paths are unsafe, so that the robot freezes, although a feasible path typically exists [16]. The self-stops in Section 6 resemble this behaviour, but they come from a learned policy behind a filter, not from a planner. Reach-avoid RL gives safety and liveness guarantees by learning a reach-avoid value function [29], a different design from post-hoc filtering.

Safety filters and shields in RL. Shields replace unsafe actions during learning [18, 30], and CBF filters have been combined with RL in several ways [31–34]. Krasowski et al. [12] compare action

replacement, projection and masking, and report that a penalty on interventions helps. Thumm et al. [14] modify the agent's action proactively to reduce fail-safe interventions, which can otherwise cause abrupt braking or aggressive steering. Banerjee et al. [15] replace the fixed backup policy of model predictive shielding, which they note often hinders task progress, with a local planner that selects recovery actions using the learned policy's value estimates. Pizarro Bejarano et al. [10] study filtering during training, a correction penalty and safe resets for PPO on quadrotor trajectory tracking; they report chattering, but their task has no obstacles and they do not analyse stalls. Yang et al. [11] filter rollouts and add a CBF reward term so that the policy can be deployed without a filter. The term penalises violations of the CBF condition by the policy's action, and the distance between the policy's action and the filtered action. Their navigation agent is a single integrator, which has no heading, so the heading geometry of Section 4 does not arise. They report success rates, but they do not analyse timeouts or stalls. Oh et al. [13] prove that RL under a perfect, least-restrictive filter, one that intervenes only when the nominal action would leave the maximal controlled-invariant set, preserves optimality. CBF-based safe RL for navigation includes [35–38]. Barriers synthesised online from range data remove the known-map assumption [39, 40].

Policy gradients under clipped or projected actions. When an action is clipped or projected before execution, many sampled actions lead to the same executed action, and therefore to the same return. Fujita and Maeda [41] study clipping to action bounds. They propose an estimator that uses the clipping and prove that it is unbiased and has lower variance than the conventional estimator, which ignores the bounds. Chou et al. [42] argue that bounded actions bias the gradient of a Gaussian policy, and propose a Beta policy. In action-constrained RL, projection-based methods can suffer from a zero-gradient problem caused by the coupling between the policy gradient and the projection [43]. Gros et al. [44] find projection onto a safe set generally disruptive for Q-learning, and give corrections that keep policy gradients unbiased. None of these works studies navigation stalls. We use them to frame, not to establish, a hypothesis about self-stops (Section 7).

Evaluation practice. RL results vary strongly between seeds [45], and few-run studies need interval estimates rather than point estimates [46]. Navigation benchmarks such as BARN [47] and Arena-Rosnav [48], and community guidelines [49], stress classical baselines and standardised metrics.

Table 1 positions this work. Entries for other papers reflect the parts we read (see LITERATURE_REVIEW.md and literature_review_gap_analysis.md for the reading depth); "n.r." means not reported in those parts, not that the paper lacks the property.

3 Problem formulation

Robot and environment. The robot state is $x_t = (p_t, \theta_t)$ with position $p_t \in [0, W]^2$, $W = 10\,\mathrm{m}$, and heading θ_t. The command is $u_t = (v_t, \omega_t) \in \mathcal{U} = [0, v_{\max}] \times [-\omega_{\max}, \omega_{\max}]$ with $v_{\max} = 1\,\mathrm{m/s}$ and $\omega_{\max} = 1\,\mathrm{rad/s}$. The simulator integrates

$$p_{t+1} = p_t + v_t\,\Delta t\,e(\theta_t), \qquad \theta_{t+1} = \theta_t + \omega_t \Delta t, \qquad e(\theta) = (\cos\theta, \sin\theta)^\top, \tag{1}$$

with $\Delta t = 0.1\,\mathrm{s}$. Obstacles are static discs with centres c_i and radii $R_i \in [0.3, 0.8]\,\mathrm{m}$; the robot is a disc of radius $r = 0.3\,\mathrm{m}$.

Table 1: Positioning. Y = yes; – = no; n.r. = not reported in the parts read; (Y) = partly.

Work	Learned policy	Filter in training	Navigation	Deadlock / liveness	Equilibria of learned policy	Feasibility oracle
Reis et al. [5]; Tan & Dimarogonas [6]	–	–	(Y)	Y	–	–
Grover et al. [7]; Jankovic et al. [8]; Chandra et al. [9]	–	–	Y	Y	–	n.r.
Krasowski et al. [12]	Y	Y	–	n.r.	–	–
Thumm et al. [14]	Y	Y	(Y)	n.r.	–	n.r.
Pizarro Bejarano et al. [10]	Y	Y	–	chattering	–	–
Oh et al. [13]	Y	Y	n.r.	n.r.	–	–
Yang et al. [11]	Y	Y	(Y)[a]	–	–	–
Banerjee et al. [15]	Y	Y	n.r.	(Y)[b]	n.r.	n.r.
Marzari et al. [36]; Guo et al. [38]	Y	(Y)	Y	–	–	–
This work	Y	Y	Y	Y	Y	Y

[a] Navigation with a single integrator, which has no heading; success rates reported, timeouts not analysed. [b] Notes that backup-policy shields often hinder task progress; navigation not stated in the abstract.

Barriers and filter. For a safety margin d (0.3 m) and a wall margin m (0.1 m), define

$$\begin{aligned} h_i(p) &= \|p - c_i\| - (R_i + r + d) && \text{(obstacle } i\text{)}, \\ h_W(p) &\in \{p_x - (r+m),\ W - p_x - (r+m),\ p_y - (r+m),\ W - p_y - (r+m)\} && \text{(walls)}. \end{aligned} \tag{2}$$

The safe set is $\mathcal{S} = \{p : h_k(p) \geq 0\ \forall k\}$. The closing rate per unit speed is $a_i(p,\theta) = e(\theta)^\top (c_i - p)/\|c_i - p\|$ for an obstacle, and $-\cos\theta, \cos\theta, -\sin\theta, \sin\theta$ for the four walls. With $\gamma = 0.5\,\mathrm{s}^{-1}$ (so $\gamma\Delta t = 0.05 \leq 1$), the filter returns

$$F(x,u) = \big(\min\{\mathrm{clip}(v), \bar{v}(x)\},\ \mathrm{clip}(\omega)\big), \qquad \bar{v}(x) = \min\Big\{v_{\max},\ \min_{k:\,a_k>0} \frac{\gamma\, h_k(p)}{a_k(p,\theta)}\Big\}. \tag{3}$$

Because every constraint bounds the single variable v, (3) is the exact solution of $\min_v (v - v^{\mathrm{nom}})^2$ subject to $a_k v \leq \gamma h_k$ and $0 \leq v \leq v_{\max}$.

Policies. A policy maps an observation o_t to a nominal command. The observation contains the noisy normalised position, $\sin\theta$ and $\cos\theta$, goal distance and bearing, a proximity feature, and 16 range readings up to 3 m. The closed loop is $x_{t+1} = f(x_t, F(x_t, \pi(o_t)))$. An episode succeeds when the goal is within 0.5 m; it ends with a collision if the robot body touches an obstacle or leaves the workspace; otherwise it times out after 200/300/400 steps (beginner/intermediate/expert).

Definition 1 (Stall classes)**.** A timeout is a *stall* if the net displacement over its last $K = 100$ steps is below $\delta = 0.10$ m. A stall is (i) *filter-blocked* (Type I) if the filter reduced the commanded speed in at least half of those steps; (ii) a *self-stop* (Type II) if, in at least half of them, the policy itself commanded a speed of at most 10^{-3} m/s; (iii) *other* otherwise. A timeout that is not a stall is *moving*.

4 Analysis of the one-step speed filter

The statements in this section follow directly from the model (1)–(3), and none of them requires new theory. We state them because they rule out some remedies and give the predictions that Section 6

tests.

Lemma 1 (Forward invariance [17])**.** *If $p_0 \in \mathcal{S}$ and $u_t = F(x_t, \cdot)$ for all t, then $h_k(p_{t+1}) \geq (1 - \gamma\Delta t)h_k(p_t)$ for every barrier k. In particular, $h_k(p_t) > 0$ for all t if $h_k(p_0) > 0$.*

Proof. By convexity of the norm, $\|p_{t+1} - c_i\| \geq \|p_t - c_i\| - \Delta t\, v_t\, a_i(p_t, \theta_t)$. If $a_i > 0$, the filter enforces $v_t a_i \leq \gamma h_i(p_t)$; if $a_i \leq 0$, the barrier cannot decrease. Wall barriers are affine in p, so the same bound holds with equality. Strict positivity follows from $\gamma\Delta t < 1$. □

Lemma 2 (Turn-rate independence)**.** *Let $\psi : \mathcal{X} \times \mathcal{U} \to [-\omega_{\max}, \omega_{\max}]$ be any map, and let $F_\psi(x, u) = (\min\{\mathrm{clip}(v), \bar{v}(x)\}, \psi(x, u))$. Then Lemma 1 holds for F_ψ.*

Proof. By (1), p_{t+1} depends only on (p_t, θ_t, v_t), and $\bar{v}(x_t)$ depends only on (p_t, θ_t). The turn rate ω_t affects only θ_{t+1}, which the filter evaluates again at step $t + 1$. The proof of Lemma 1 therefore applies at every step, whatever ψ returns. □

Remark 1. Lemma 2 is the discrete-time counterpart of a known structural fact. In continuous time, the position barrier has relative degree two with respect to ω, which is the situation high-order CBFs address [21]. It has two practical consequences here. First, the one-step filter can never *steer* the robot away from an obstacle: only the policy, or a heading modification ψ, can do so. Second, heading-level remedies for deadlock, such as a tangential turn when the speed limit is active, inherit the safety guarantee without further proof.

Proposition 1 (Collapse of the admissible headings)**.** *Fix p with $0 \leq h_i(p) \leq \varepsilon$ for an obstacle i, and a speed $\underline{v} > 0$. Every heading θ with $\bar{v}(p, \theta) \geq \underline{v}$ satisfies $a_i(p, \theta) \leq \gamma\varepsilon/\underline{v}$; that is, the angle between $e(\theta)$ and the direction to c_i is at least $\arccos(\min\{1, \gamma\varepsilon/\underline{v}\})$. As $\varepsilon \to 0$, this set of headings approaches the closed half-circle $\{\theta : a_i(p, \theta) \leq 0\}$.*

Proof. If $a_i(p, \theta) > 0$, then $\bar{v} \leq \gamma h_i / a_i \leq \gamma\varepsilon/a_i$, and $\bar{v} \geq \underline{v}$ requires $a_i \leq \gamma\varepsilon/\underline{v}$. If $a_i \leq 0$, the inequality holds trivially. □

Corollary 1 (Stall bound)**.** *If, over K consecutive steps, $h_i(p_t) \leq \varepsilon$ and $a_i(p_t, \theta_t) \geq \bar{a} > 0$, then the robot moves at most $K\Delta t\, \gamma\varepsilon/\bar{a}$, whatever speed the policy commands. For a head-on approach ($a_i \equiv 1$) at the speed limit, $h_i(p_{t+1}) = (1 - \gamma\Delta t)\, h_i(p_t)$ holds with equality, so the barrier decays geometrically and the robot approaches, but never reaches, the boundary.*

Proof. The displacement per step is $v_t\Delta t \leq \bar{v}\Delta t \leq \gamma\varepsilon\Delta t/\bar{a}$. For straight motion toward the centre, the convexity bound in Lemma 1 is tight, giving the stated equality. □

Lemma 3 (Test-time speed limits at near-zero speed commands)**.** *Let F' be any filter whose executed speed satisfies $0 \leq v^{\mathrm{exec}} \leq \mathrm{clip}(v^{\mathrm{nom}})$, and let $\underline{v} \geq 0$. At every step where the deterministic policy commands $\pi_v(o) \leq \underline{v}$, the executed speed under F' is at most $\underline{v}$, as it is without a filter; if $\pi_v(o) \leq 0$, it is exactly 0. In particular, at the steps of Definition 1 where the policy commands at most $\underline{v} = 10^{-3}$ m/s, no test-time speed limit can make the robot move faster than it would without a filter. At these steps, only a different command from the policy can produce motion.*

Proof. $0 \leq v^{\mathrm{exec}} \leq \mathrm{clip}(\pi_v(o)) \leq \max\{0, \underline{v}\} = \underline{v}$, and $\mathrm{clip}(\pi_v(o)) = 0$ if $\pi_v(o) \leq 0$. □

Lemma 3 concerns only the steps at which the policy commands a near-zero speed. It says nothing about:

- the other steps of a stall, at which the policy may command a larger speed;

- how a different speed limit at earlier steps would change the trajectory, and with it the later observations;
- the effect of the filter during training.

Proposition 1 and Lemma 3 give testable predictions. Filter-blocked stalls should occur at $h \approx 0$ with headings within 90° of the obstacle direction. Self-stops should occur at $h > 0$. At their stopping steps, the speed should be set by the policy's command, not by the test-time speed limit.

5 Experimental setup

Agents. PPO [50] with separate 256×256 tanh actor and critic networks, a diagonal Gaussian policy whose samples are clipped by the environment, generalised advantage estimation and bootstrapping at time-limit truncations. Training ran for 10^6 steps over a three-stage curriculum (beginner, intermediate, expert; advance at 70 % rolling success). Five seeds (0–4) were trained without the filter and five with the filter in the loop. In the filtered runs, the buffer stores the policy's own sample, as in [10]. The checkpoint with the highest success on 60 validation episodes (20 per stage), on seeds disjoint from the test seeds, was kept. One hyperparameter set was used for both variants and was not tuned for the filtered setting. All ten runs reached the expert stage within 43k–76k steps.

Baselines. A go-to-goal heading controller, which ignores obstacles, and a hand-tuned artificial potential field on the range readings [28]. A DWA-style local planner [51] uses the same observation (16 range readings, goal distance and bearing; no map). It scores 66 constant-command rollouts of 1.5 s by progress, heading, clearance and speed, and rejects those whose predicted clearance falls below a margin. Without inertia, the dynamic window is the whole action box. Its weights were selected from 12 configurations on 100 validation scenarios (seeds from $1.3 \cdot 10^6$) by a rule fixed in advance, and then frozen. PPO with the correction penalty of [10] is evaluated as a remedy (Section 6.10). [**EVIDENCE REQUIRED:** PPO-Lagrangian [52] and MPC with a DT-CBF [20] are still missing.]

Test scenarios. 100 seeded scenarios per stage (seed bases $2 \cdot 10^6 + 10^5 s$). The expert stage has 8–12 obstacles. A grid oracle (5 cm, 8-connected Dijkstra on the free space inflated by $r + d$) found a margin-keeping path in all 300 scenarios. Two properties of the generator must be kept in mind:

- The goal is clipped to $[1.5, 8.5]^2$, so 53 of the 100 expert goals are closer than the configured minimum of 5 m (maximum 8.9 m).
- Three scenarios (two beginner, one expert) start inside the goal region and count as successes at step 1 for every controller.

The 0.3 m margin and the sampled-action evaluation were chosen after inspecting these test scenarios, so the test set is not a clean held-out set for those two choices.

Fresh scenarios and pre-registration. To address this, we evaluated the agents on 500 expert scenarios that had never been used (seeds $9 \cdot 10^6$ to $9 \cdot 10^6 + 499$). The hypotheses, decision rules and analysis were committed to the repository before this evaluation was run. The scenarios come from the same generator, so the goal-clipping caveat still applies.

Corrected generator. A second pre-registered evaluation (E3) removes the clipping. The generator redraws start, heading and goal together until the goal lies inside the workspace at the requested distance. The start stays in the original square, so only the goal-distance artefact changes. It uses 1,000 new scenarios per stage (intermediate: seeds from $7.5 \cdot 10^6$; expert: from $7.6 \cdot 10^6$), with mean and sampled actions. An exploratory dense variant uses 500 expert scenarios with 15–25 requested obstacles (seeds from $6 \cdot 10^6$). In the corrected sets, expert start–goal distances range from 5.0 to 9.0 m (median 5.8 m).

Heading escape. As a test-time intervention, we add to the filter a turn-rate modification ψ. When the filter reduces the commanded speed, the turn rate becomes $\mathrm{clip}(\omega^{\mathrm{nom}} + \kappa\, s\, \omega_{\max})$. Here $s \in \{-1, +1\}$ is the turn direction that increases the angle between the heading and the direction of the binding barrier (towards its tangent); if that angle is zero, s points to the goal side. By Lemma 2, the safety guarantee holds for every κ. The value $\kappa = 0.5$ was selected from $\{0.5, 1.0\}$ by success on 100 validation scenarios (seeds $1.2 \cdot 10^6$ onward) using a rule fixed in advance; it was then frozen. The agents are not retrained.

Checkpoint selection. To test whether self-stops are an artefact of selecting early checkpoints, we also evaluated the final (10^6-step) checkpoints of the five filter-trained runs on the fresh scenarios. These checkpoints are local training outputs and are not yet in the public repository; their SHA-256 hashes are recorded.

Conditions and classification. Each PPO agent was evaluated with the filter off and on, using the policy mean. Each was additionally evaluated with the filter on and actions sampled from the policy, seeded per episode. Every timeout with the filter on was replayed deterministically from its seed and classified by Definition 1. This classification is a post-hoc analysis made after the benchmark had been run.

Statistics. For PPO, the unit of replication is a training seed. Seed-level 95 % t-intervals over five seeds describe variation between training runs on the fixed scenario set. Comparisons between conditions use a paired hierarchical bootstrap (10,000 replicates). Scenarios are resampled jointly for both conditions, because all conditions share the same scenarios, and seeds are resampled within each condition. Proportions for the deterministic baselines use Wilson intervals over episodes. A two-way decomposition of episode success into seed, scenario and residual sums of squares quantifies the two sources of uncertainty. No multiplicity correction was applied, because the contrasts are reported as estimates with intervals and are not used as a family of hypothesis tests. With five seeds, the bootstrap underestimates seed variance, so the intervals should be read as approximate.

Reproducibility. The repository contains the committed checkpoints, and regenerating the benchmark from them reproduces the per-episode files byte for byte. All analysis scripts read only stored outputs; Section 9 lists them.

6 Results

6.1 Safety and success

With the filter on, none of the 3,600 test episodes (all controllers and stages) had a collision, and the smallest obstacle clearance was the 0.30 m margin (Table 2). The filter's cost in success depends

Table 2: Expert stage, 100 test scenarios. PPO: mean over five training seeds with 95 % t-interval over seeds (500 episodes); baselines: Wilson interval over 100 episodes. Sources: `results/benchmark/summary.csv`, `results/stochastic_eval/summary.json`.

Condition	Success (%)	Collision (%)	Timeout (%)	Interventions (%)
PPO trained w/o filter, filter off	94.6 [91.7, 97.5]	1.0	4.4	–
PPO trained w/o filter, filter on	65.0 [61.5, 68.5]	0	35.0	58.0
PPO trained w/o filter, filter on, sampled	70.0 [66.3, 73.7]	0	30.0	n.r.
PPO trained with filter, filter on	61.0 [55.4, 66.6]	0	39.0	41.5
PPO trained with filter, filter on, sampled	76.4 [68.2, 84.6]	0	23.6	n.r.
Go-to-goal, filter off	57.0 [47.2, 66.3]	43.0	0	–
Go-to-goal, filter on	46.0 [36.6, 55.7]	0	54.0	67.2
Potential field, filter off	73.0 [63.6, 80.7]	0	27.0	–
Potential field, filter on	72.0 [62.5, 79.9]	0	28.0	40.7

on the controller. For PPO trained without the filter, expert success fell from 94.6 % to 65.0 % when the filter was added at test time, a paired difference of 29.6 points [21.6, 38.4]. The agent trained with the filter reached 61.0 %. The paired difference to the unfiltered-trained agent with the filter on was −4.0 points [−10.8, 3.2] with mean actions, and +6.4 points [−1.6, 15.2] with sampled actions. Neither comparison shows that training with the filter improves success.

6.2 Two stall mechanisms

Figure 1 and Table 3 classify every timeout with the filter on.

- **PPO trained without the filter.** Filter-blocked stalls dominate: 141 of 175 expert timeouts and 68 of 81 intermediate timeouts, with mean actions. In these stalls the policy commands full speed (median 1.0 m/s) while the turn rate changes sign a median of 70 times in 100 steps.
- **PPO trained with the filter.** Self-stops dominate: 122 of 195 expert timeouts and 68 of 100 intermediate timeouts. The policy commands zero speed at a median barrier value of 0.41 m (interquartile range 0.16–0.72 m), well before the keep-out boundary. In 89 of the 122 expert self-stops, the goal lies within 45° of the direction of the nearest obstacle (median 26°), and the robot barely turns (median $|\omega|$ 0.04 rad/s).
- **Baselines.** The two classical controllers bracket the PPO behaviour. Go-to-goal, which always commands forward speed toward the goal, fails only by filter-blocking (54 of 54 expert timeouts). The potential field never stalls against the filter: it oscillates in local minima (22 “moving” timeouts) or stops (6).

The taxonomy is insensitive to its thresholds. Varying the displacement threshold (0.05, 0.10 and 0.20 m) and the share threshold (30, 50 and 70 %) changes the expert counts by at most 3 (filter-blocked) and 3 (self-stop) for either agent type. Varying the window length K over 50, 100, 150 and 200 steps, with the displacement threshold scaled to the window, leaves the counts of the agent trained without the filter unchanged (141 and 34). For the agent trained with the filter, the filter-blocked count ranges over 63–69 and the self-stop count over 119–122.

6.3 The geometry of filter-blocked stalls matches the analysis

Proposition 1 predicts that a robot blocked at the boundary heads less than 90° from the obstacle direction. The data confirm this (Figure 2a).

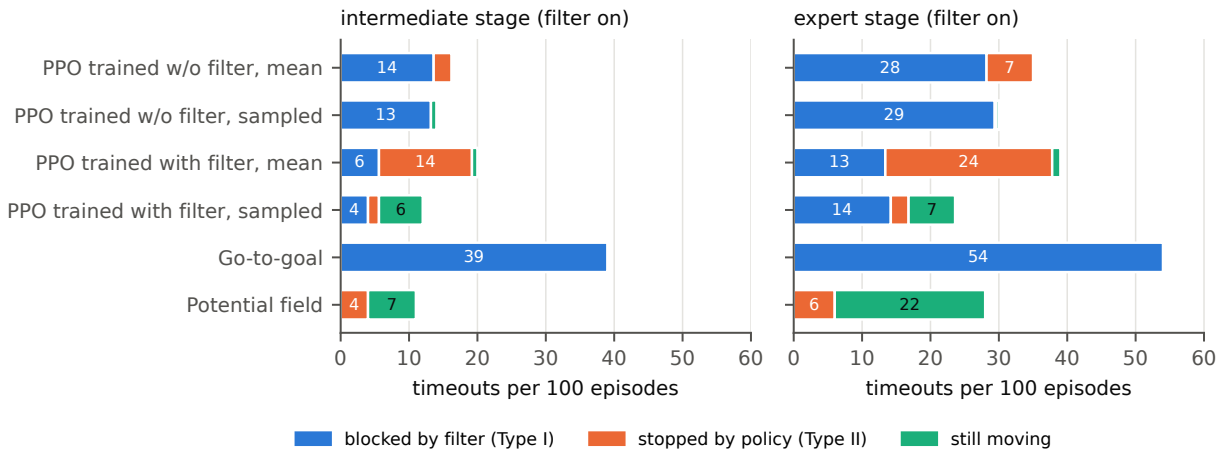


Figure 1: Timeouts per 100 episodes with the filter on, by stall class (Definition 1). PPO: five seeds × 100 scenarios; baselines: 100 scenarios. Source: `research/audit_outputs/stall_taxonomy_all.csv`.

Table 3: Timeout classes with the filter on (counts; PPO over 500 episodes per row, baselines over 100). Source: `research/audit_outputs/stall_taxonomy_all.csv`.

	Intermediate			Expert		
Condition	Type I	Type II	moving	Type I	Type II	moving
PPO trained w/o filter, mean	68	13	0	141	34	0
PPO trained w/o filter, sampled	66	0	4	147	1	2
PPO trained with filter, mean	28	68	4	67	122	6
PPO trained with filter, sampled	20	8	32	71	13	34
Go-to-goal	39	0	0	54	0	0
Potential field	0	4	7	0	6	22

- 617 of the 618 filter-blocked stalls of all PPO conditions have a heading below 90°; the remaining one is at 91.3°. The median is 66.8°, so the robots are typically not facing the obstacle head-on.
- The obstacle barrier at the end of these stalls is at most 4.2 mm, consistent with the geometric decay in Corollary 1.
- In all 618 stalls, the active barrier is an obstacle: no wall barrier is below 1 mm.

Self-stops, in contrast, are spread over barrier values from 0 to 1.4 m (Figure 2b). This is consistent with Lemma 3: at these stops, the speed is set by the policy's command, not by the test-time speed limit. Whether the filter shaped this behaviour during training is discussed in Section 7.

6.4 Training with the filter shifts the mechanism; sampling removes self-stops

Paired comparisons (Figure 3) quantify both effects.

- **Training with the filter.** On the expert stage, it lowered the filter-blocked rate by 14.8 points [5.6, 24.4] and raised the self-stop rate by 17.6 points [8.2, 27.0]. On the intermediate stage, the corresponding changes were −8.0 [−14.8, −1.8] and +11.0 [5.6, 17.2].

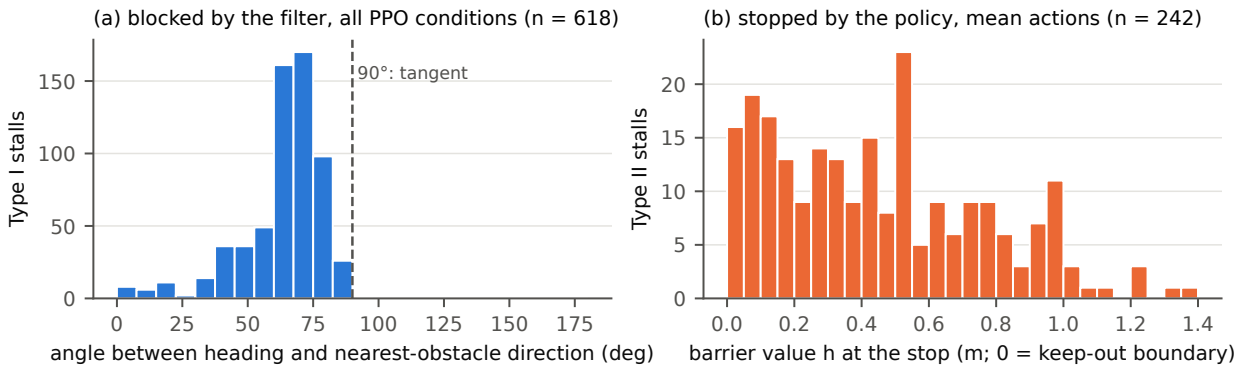


Figure 2: (a) Heading relative to the nearest obstacle at the end of filter-blocked stalls (all PPO conditions, both stages); the dashed line marks 90° (tangent). (b) Barrier value at self-stops (PPO, mean actions). Source: `research/audit_outputs/stall_taxonomy_all.csv`.

- **Sampled actions** (agent trained with the filter). The expert self-stop rate fell by 21.8 points [12.4, 31.6], while the filter-blocked rate did not change (+0.8 [−7.6, 10.0]).
- **Success.** Sampling raised expert success by 15.4 points [7.8, 24.0] for the filter-trained agent, against 5.0 [0.4, 10.4] for the agent trained without the filter.

On these scenarios, stochastic deployment acted on the learned mechanism only. The filter-blocked stalls persist, as Proposition 1 suggests they should for any policy that keeps a heading within 90° of the obstacle. On the corrected generator with longer goal distances, sampling also *added* filter-blocked stalls (Section 6.8).

6.5 Sources of uncertainty

The decomposition of expert success shows how much the test scenario matters. Depending on the condition, the scenario term accounts for 25–86 % of the sum of squares, against under 2 % for the seed term. Seed-only intervals such as those in Table 2 therefore describe reproducibility of training on these 100 scenarios; they understate the uncertainty about the scenario distribution. An approximate fit of interval width to the number of seeds and scenarios suggests that about 10 seeds and 1,000 scenarios per stage would give half-widths near 4–5 points for differences of this kind. Beyond about 1,000 scenarios, more seeds help more than more scenarios.

6.6 Replication on previously unused scenarios

On the 500 fresh expert scenarios (5,000 episodes per condition, mean actions), the taxonomy and both training contrasts replicate (Table 4).

- Filter-blocked stalls make up 709 of the 792 timeouts of the agents trained without the filter.
- Self-stops make up 577 of the 903 timeouts of the agents trained with it.
- Training with the filter lowered the filter-blocked rate by 16.5 points [10.1, 22.8] and raised the self-stop rate by 19.8 points [12.8, 26.1].

Both effects are close to those on the original test set. In this larger sample, expert success of the filter-trained agents was also lower than that of the agents trained without the filter, by 4.4 points

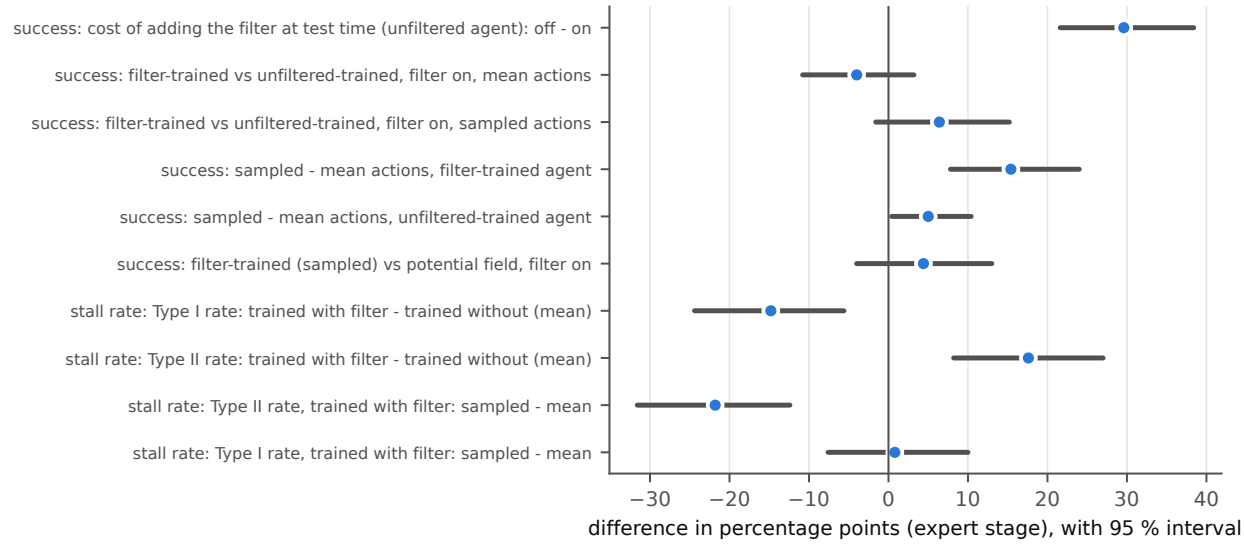


Figure 3: Paired differences (expert stage) with 95 % paired hierarchical bootstrap intervals. Success differences: `research/audit_outputs/paired_comparisons_expert.csv`; stall-rate differences: `paper_draft/generated/stats.json`.

Table 4: Pre-registered evaluation on 500 previously unused expert scenarios (five agents per row, 2,500 episodes, mean actions). κ: heading escape (0 = plain filter). Source: `research/experiments/lightweight/results/summary.json`.

Agents	κ	Success (%)	Timeouts	Type I	Type II	moving
trained w/o filter	0	68.3	792	709	81	2
trained w/o filter	0.5	75.6	611	523	85	3
trained with filter	0	63.9	903	297	577	29
trained with filter	0.5	64.0	901	244	615	42
trained with filter, final checkpoints	0	59.1	1023	123	870	30

[0.2, 8.5]. This comparison was exploratory, not one of the pre-registered tests. There were no collisions in the 10,000 fresh-scenario episodes, and the minimum clearance was 0.30 m.

6.7 Self-stops are not an artefact of checkpoint selection

The final checkpoints of the filter-trained runs have more self-stops than the selected checkpoints: +11.7 points [3.0, 20.8]. They have fewer filter-blocked stalls (−7.0 [−13.0, −1.5]) and lower success (−4.8 [−8.8, −1.2]). The increase comes from the three seeds whose selected checkpoint was from early in training (0.1–0.3M steps). Self-stops in these seeds rose from 139, 51 and 105 to 205, 174 and 206. In the seed whose best checkpoint is its final one, the results are identical. Longer training with the filter therefore strengthens the self-stopping behaviour rather than removing it.

6.8 Corrected generator: honest goal distances

Table 5 and Figure 4 summarise the pre-registered E3 evaluation (46,000 episodes, plus 6,500 in the dense variant; all with the filter on, no collisions, minimum clearance 0.30 m).

Table 5: Corrected generator (E3), 1,000 scenarios per stage, filter on. PPO: mean over five seeds with 95 % t-interval over seeds; baselines: Wilson interval over episodes. Source: `research/experiments/round2/results/summary.json`.

	Expert success (%)		Intermediate success (%)	
Controller	mean actions	sampled	mean actions	sampled
PPO trained w/o filter	43.1 [38.2, 48.1]	53.2 [49.3, 57.0]	73.8 [70.1, 77.6]	80.1 [77.4, 82.7]
PPO trained with filter	37.2 [29.0, 45.5]	55.6 [45.7, 65.4]	66.0 [59.1, 73.0]	78.5 [70.9, 86.1]
Potential field	57.0 [53.9, 60.0]	–	83.5 [81.1, 85.7]	–
DWA-style (frozen)	15.3 [13.2, 17.7]	–	36.9 [34.0, 39.9]	–
DWA-style, margin 0.35 m (exploratory)	43.6 [40.6, 46.7]	–	n.r.	–
Go-to-goal	9.8 [8.1, 11.8]	–	29.4 [26.7, 32.3]	–

- **Success drops sharply.** With goals at their requested distance, expert success with mean actions is 37.2 % (filter-trained) and 43.1 % (trained without the filter). On the clipped fresh scenarios it was 63.9 % and 68.3 %. Success also falls with distance: 38.8 % at 5–7 m against 21.1 % at 7–9 m for the filter-trained agents. The two sets differ in scenarios as well as in generator, so this comparison is descriptive. A large part of the expert success on the original generator nevertheless comes from goals that clipping moved closer.

- **The training contrasts replicate.** Training with the filter lowered the expert filter-blocked rate by 29.7 points [20.6, 37.7] and raised the self-stop rate by 34.1 points [22.9, 42.9]. Sampled actions lowered the self-stop rate of the filter-trained agents by 35.8 points [25.7, 43.5].

- **One null does not replicate.** On the original test set, sampling did not change the filter-blocked rate of the filter-trained agents (+0.8 [−7.6, 10.0]). Here it raised that rate by 7.3 points [2.5, 14.3], from 21.0 % to 28.3 %: part of the released self-stops end at the filter boundary. The pre-registered hypothesis that all four contrasts behave as before is therefore only partly supported in the expert stage (3 of 4 criteria). In the intermediate stage, all four criteria hold.

- **Training with the filter does not raise success.** The expert difference is −5.9 points [−12.5, 0.3]; the intermediate difference is −7.8 [−13.2, −2.3].

- **The untuned potential field is the strongest controller on these sets** (expert 57.0 % [53.9, 60.0]). Its timeouts are mostly "moving", not stalls. The exploratory dense variant (15–25 obstacles) lowers every controller's success, with the same ranking.

6.9 A classical planner stalls behind the filter too

The DWA-style planner shows that filter-blocked stalls are not specific to learned policies (Figure 5).

- **Without the filter**, it solved 96 % [90.2, 98.4] of the 100 expert test scenarios, with no collisions. Its minimum clearance was only 0.027 m, far inside the filter's 0.30 m margin.

- **With the filter**, its expert success fell to 48 % [38.5, 57.7], and *every* timeout was filter-blocked. The same held on the 500 fresh scenarios (50.0 %; 250 of 250 timeouts) and on the corrected generator (15.3 %; 847 of 847). All 12 tuning configurations reached 37–39 % validation success, so the weights hardly mattered. The planner steers into the band the filter forbids and is held there.

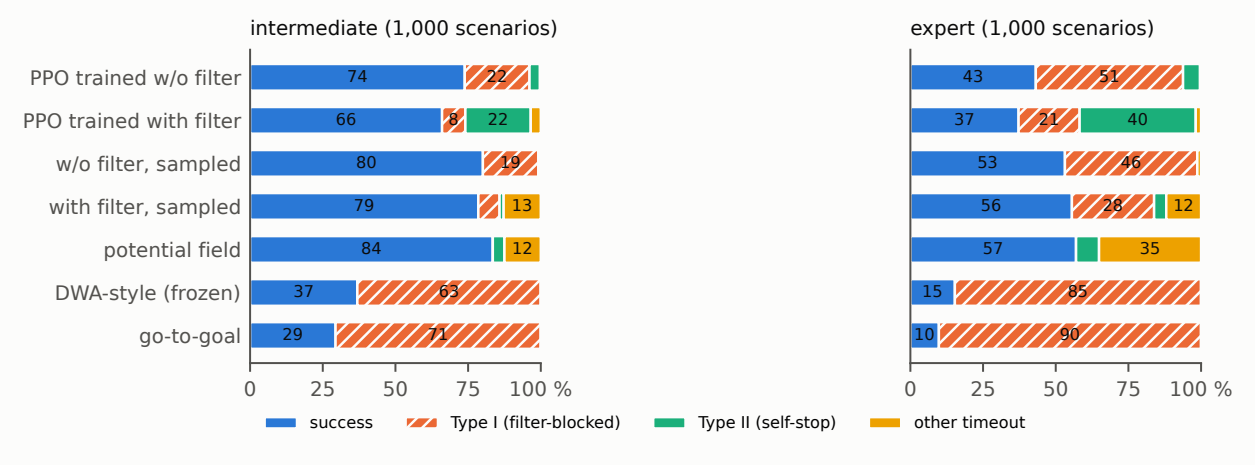


Figure 4: Outcome composition on the corrected generator (E3), filter on, 1,000 scenarios per stage (PPO: five seeds pooled). Numbers are percentages of episodes. Source: `research/experiments/round2/results/e3_*/episodes.csv`.

- **Exploratory follow-up.** This check was added after these results and committed before it was run. Raising the planner's own admissibility margin to 0.35 m, just above the filter margin, removed *all* filter-blocked stalls in all three sets. Success rose to 63 % [53.2, 71.8] (test), 67.8 % [63.6, 71.7] (fresh) and 43.6 % [40.6, 46.7] (corrected generator). The remaining timeouts were stops in which the planner found no admissible rollout and turned in place. A margin of 0.45 m gave fewer successes and more such stops.

For a classical planner, filter-blocking is therefore a mismatch between the planner's margin and the filter's, which a filter-aware design avoids. What remains is the planner's own local-minimum problem. This mirrors the learned agents: training with the filter removes most filter-blocked stalls, and self-stops take their place.

6.10 Remedies

Heading escape (test time, frozen agents). With $\kappa = 0.5$ (Table 4), the filter-blocked rate of the agents trained without the filter fell by 7.4 points [4.8, 10.2], and their success rose by 7.2 points [5.0, 9.7]. The reduction held in every seed. Self-stops of the filter-trained agents did not change (+1.5 [−0.8, 3.9]), nor did their success (+0.1 [−2.2, 2.4]). This is expected. The escape is triggered only when the filter reduces the commanded speed, which never happens while the policy commands zero speed (Lemma 3). A heading change triggered during a stop could in principle change the policy's observation and release it, but we did not test this. There were no collisions, as Lemma 2 guarantees. The effect is real but small. It is below the 10-point threshold we fixed in advance, and 523 of the remaining 611 timeouts of these agents are still filter-blocked. A one-step heading rule is therefore not enough to resolve Type I stalls; it is a lower bound on what filter-level heading changes can do.

Correction penalty (E1a, pre-registered). We trained PPO with the correction-penalty term of [10] ($\alpha = 1$, our one-step filter, no safe resets) and an identical control: five seeds and 300k steps per arm, filter in the loop. The primary endpoints were fixed before training: the deadlock rate (filter-blocked + self-stop + other stalls) and success, both with the policy mean on 500 previously

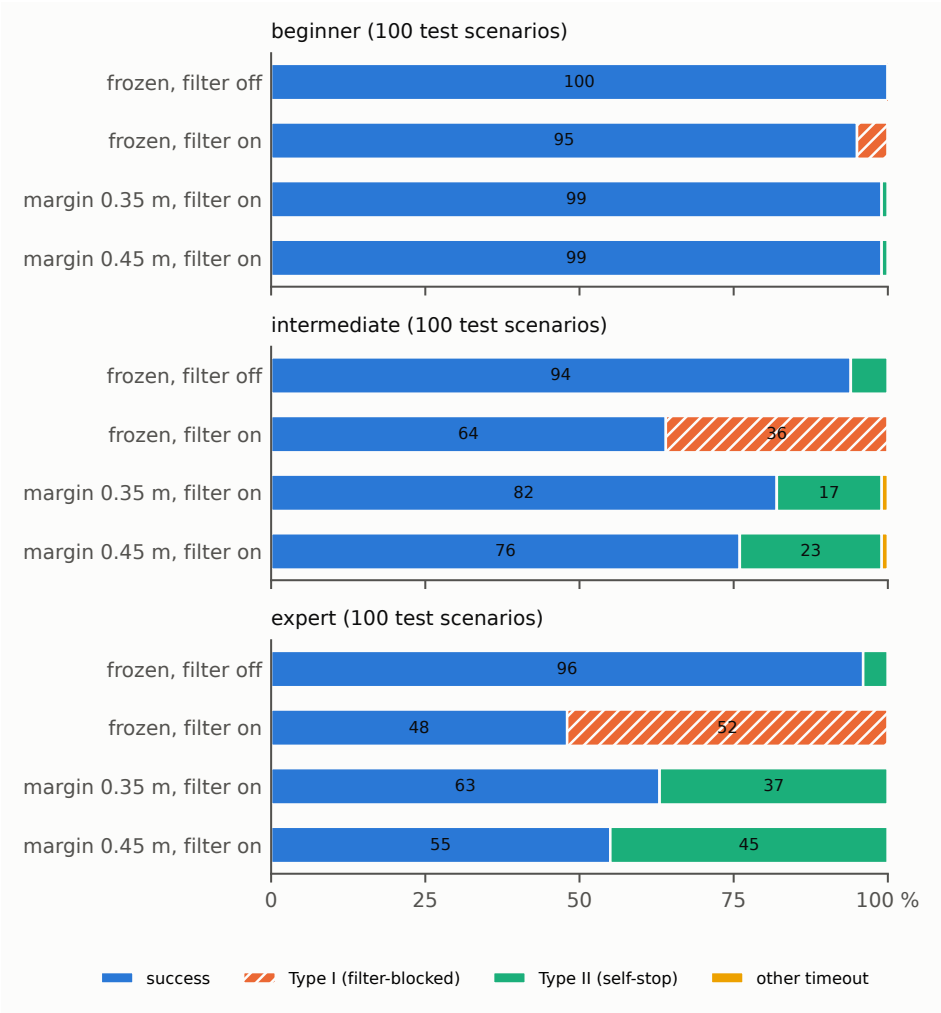


Figure 5: DWA-style planner on the 100 test scenarios per stage: frozen weights with the filter off and on, and the exploratory filter-aware margins (filter on). Numbers are percentages of episodes. Source: `research/experiments/round2/results/dwa_*`.

unused expert scenarios (seeds from $5 \cdot 10^6$), with a Holm adjustment over the two tests. Penalty minus control:

- deadlock rate: -5.1 points $[-11.6, 1.7]$ (Holm-adjusted $p = 0.28$);
- success: $+0.5$ points $[-5.8, 6.3]$ ($p = 0.87$);
- collisions: none in any of the 24,000 evaluation episodes.

By the pre-registered rule (a reduction of at least 10 points with an interval excluding 0), the penalty does not resolve deadlock in this setting at this budget. The secondary contrasts were not adjusted for multiplicity. With mean actions, they show what changed (Figure 6): the self-stop rate fell by 19.7 points [6.9, 32.2] and the filter-blocked rate rose by 14.0 points [2.4, 26.6]. In two seeds, self-stops almost vanished (0.0 % and 4.4 %), but filter-blocked stalls rose to 40 % and 33 %. The penalty thus appears to trade one mechanism for the other rather than remove deadlock. With five seeds and a large spread between them, this is suggestive, not established. With sampled actions, the

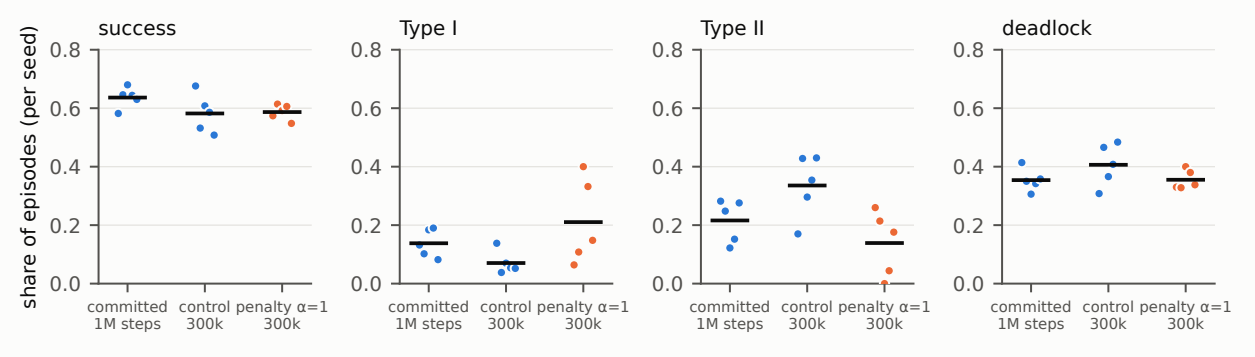


Figure 6: E1a, 500 fresh expert scenarios, mean actions: per-seed rates (dots) and their means (bars) for the committed 10^6-step agents, the 300k-step control and the 300k-step correction-penalty agents ($\alpha = 1$). Source: `research/experiments/e1a/eval/episodes.csv`.

arms did not differ. The penalty agents were 5.0 points [0.8, 9.1] less successful than the committed agents, which were trained for 10^6 steps. Only $\alpha = 1$ was tested.

Removing either candidate cause of self-stops (E4, pre-registered). We trained two further arms, identical to the E1a control except for one change each (five seeds, 300k steps): a Beta policy [42], whose samples always lie inside the action bounds, so no speed sample is clipped to zero; and a reward without the risk penalty. The primary endpoint was the policy-side stall rate (self-stops plus other stalls in which the filter rarely limited the speed), with the policy mean on 500 previously unused expert scenarios (seeds from $6.5 \cdot 10^6$). The criterion was a reduction of at least 10 points with a Holm-adjusted interval excluding 0. The control's policy-side stall rate was 33.8 % (seed SD 9.3).

- **Beta policy:** 0.0 % in every seed, a difference of −33.8 points [−41.3, −25.2].
- **No risk penalty:** 2.2 %, a difference of −31.6 points [−39.4, −22.9] (both Holm-adjusted $p < 0.001$).

Both criteria hold, so each change on its own is sufficient to remove the self-stops. Neither of the two candidate explanations is *the* cause. Neither change removed the deadlock, however (Figure 7). The filter-blocked rate rose by 33.6 points [27.3, 40.1] (Beta) and 26.2 points [21.0, 31.4] (no risk penalty). The Beta agents' stalls end exactly at the boundary (median barrier 0.000 m) with a forward speed command (median 0.52 m/s), so they are not hidden stops. Total deadlock was 39.7 % (control), 39.4 % (Beta) and 34.2 % (no risk penalty). Success did not change for the Beta policy (−0.2 [−6.3, 5.5]) and rose by 6.2 points [1.2, 11.2] without the risk penalty. There were no collisions in 15,000 episodes. As stated in advance, the Beta policy also changes exploration, the risk-free reward also removes an incentive to keep clearance, and neither arm was tuned.

7 Discussion

Two different problems. Most discussions of filter-induced deadlock treat it as one problem of the filter [5, 6]. Our data show two different problems, and which one dominates depends on the training setup. A policy that never saw the filter keeps pushing at the boundary. It is held there because, by Proposition 1, no heading within 90° of the obstacle admits motion. This is the

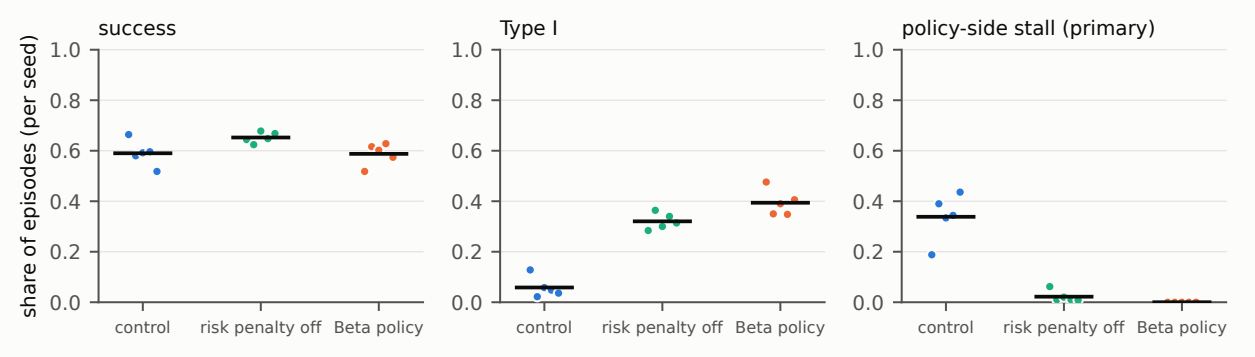


Figure 7: E4, 500 fresh expert scenarios, mean actions: per-seed rates (dots) and means (bars) for the control (the E1a control), a reward without the risk penalty, and a Beta policy (all 300k steps, filter in training). Source: `research/experiments/round2/results/e4_mean/episodes.csv`.

learned-policy counterpart of the undesired boundary equilibria of CBF-QPs. A policy trained with the filter is blocked less often, but it mostly stops on its own, typically when an obstacle lies between it and the goal. By Lemma 3, at the steps where it commands a near-zero speed, no test-time speed limit can make it move. Whether the filter contributed to this behaviour during training is a separate question, discussed next.

Why do filter-trained policies stop? We separate what is observed from what is hypothesised.

- **Observed.** Self-stops occur at a positive barrier value (median 0.41 m), mostly when the goal lies roughly in the direction of the nearest obstacle (89 of 122 expert self-stops within 45°). They are much less frequent in agents trained without the filter: 34 vs. 122 of 500 expert episodes on the original scenarios, and 81 vs. 577 of 2,500 on the fresh scenarios. Sampled actions remove most of them. A heading escape that is triggered by speed reductions leaves them unchanged. They are more frequent at the final training checkpoints than at the selected ones.
- **Not established by these observations.** The cause. None of them is a causal test. The training interventions of E4, reported below, are.

We consider two candidate explanations. The first is *action aliasing* in the filtered environment. Every sampled speed $v \leq 0$ is executed as 0, and every speed above the limit is executed as the limit. Within these sets the executed action, and therefore the return, does not depend on the sampled value, so the return carries no information about which of these samples to prefer. Related effects of clipping and projection on policy gradients are documented: variance from ignoring action bounds [41], bias of Gaussian policies under bounded actions [42], zero gradients under projection layers [43], and the need for corrections when projecting onto a safe set [44]. Our setting is closest to clipping after sampling, as studied by Fujita and Maeda, because the filter acts in the environment and the buffer stores the policy's own sample. Those analyses concern gradient variance and bias, not navigation behaviour, so they make aliasing plausible but do not show that it causes self-stops. Under this explanation, the mean could drift into the stopping region near obstacles, where forward commands were often truncated during training, and sampling would occasionally leave it. Both points are consistent with the observations, but not specific to this explanation. The second explanation is the reward: the non-potential-based risk penalty may make stopping near obstacles attractive. The two explanations are not exclusive.

Hypothesis 1. Type II stops arise mainly from aliasing rather than from the reward. Then a policy whose distribution respects the action bounds (for example a Beta policy [42]), or a clipping-aware gradient estimator [41], should reduce them, while removing the risk penalty should not.

E4 tested this hypothesis, and the data reject its second part. A Beta policy removed the self-stops, and so did removing the risk penalty. The diagnostics are consistent with both routes. In the committed agents, every self-stop had a raw speed command at or below zero in at least half of its last 100 steps (median -0.20), which clipping maps to exactly zero. In 71 % of them, however, the risk term was zero at the stopping position.

The stronger result is what happens instead. Every intervention that removed self-stops replaced them with filter-blocked stalls at the boundary: the Beta policy, the risk-free reward and, partly, the correction penalty. Total deadlock barely changed. The same exchange appears between agents trained without and with the filter, and, in reverse, for a planner that keeps its own margin above the filter's (Section 6.9). This suggests a reading that we have not tested directly. Near an obstacle that lies between the robot and the goal, these policies have no way to make progress. Stopping early and being held at the boundary are then two ways of failing, and the training details decide which one appears. If so, removing self-stops is not a remedy. A remedy has to supply the missing manoeuvre around the obstacle, for example by planning or by a multi-step escape.

Relation to theory on filtered training. Oh et al. [13] prove that RL under a perfect, least-restrictive filter preserves optimality. Such a filter intervenes if and only if the nominal action would make the state leave the maximal controlled-invariant subset of the safety constraint. The filter studied here is not least-restrictive in this sense, under the following assumption and model.

- **Assumption.** The safety constraint is the margin set $\mathcal{S}$ that our filter enforces.
- **Controlled-invariant set.** In the model (1), $v = 0$ is admissible and keeps the position fixed. Every state whose position lies in $\mathcal{S}$ can therefore be kept in $\mathcal{S}$ indefinitely, so the maximal controlled-invariant set consists of all such states. A least-restrictive filter then intervenes only when the next position would leave $\mathcal{S}$.
- **Exit condition.** The bound used in the proof of Lemma 1, $h_k(p_{t+1}) \geq h_k(p_t) - \Delta t\, v_t\, a_k(p_t, \theta_t)$ (with equality for walls), shows that leaving $\mathcal{S}$ through a closing barrier k ($a_k > 0$) in one step requires $v_t\, a_k > h_k/\Delta t$. This is a necessary condition, not a sufficient one. For an oblique approach to an obstacle, the exact speed at which the next position leaves $\mathcal{S}$ can be larger.
- **Comparison.** Our filter limits the same quantity to $v_t\, a_k \leq \gamma h_k$. Both bounds are in m/s, and their ratio is $\gamma \Delta t = 0.05$. For a given state, heading and closing barrier, the filter's speed cap $\gamma h_k / a_k$ is therefore at most one twentieth of the speed at which a least-restrictive filter would have to intervene for that barrier. Because the exit condition is necessary and $v \leq v_{\max}$, a least-restrictive filter can act on barrier k only where $h_k < a_k v_{\max} \Delta t \leq 0.1\,\mathrm{m}$.

If the safety constraint were collision avoidance without the margin, the maximal controlled-invariant set would be larger, and a least-restrictive filter would intervene even less. Oh et al.'s result therefore does not apply directly to our filter, and it does not tell us how much of the gap the filter itself causes. Our reading that much of the remaining gap is a learning problem rests on our own evidence instead:

- the oracle finds a margin-keeping path in every test scenario;
- most stalls of filter-trained agents are self-stops at a positive barrier value, where the speed is set by the policy's near-zero command rather than by the test-time speed limit (Lemma 3).

This does not exclude that the filter shaped the stopping behaviour during training, which is the aliasing explanation above. Whether a less restrictive filter, for example a larger γ, would reduce the filter-blocked stalls is open.

Implications for remedy design. Lemma 2 means that heading-level remedies come at no cost to the one-step guarantee. The simple escape tested here helps, but only modestly. A heading rule that looks one step ahead cannot plan a way around an obstacle. Multi-step or planning-level recoveries are the natural next candidates, for example task-aware recovery planning as in [15]. By Lemma 3, a remedy acting only on the test-time speed limit, such as a less conservative γ, cannot produce motion at steps where the policy commands a near-zero speed. It could still change earlier parts of the trajectory, which we did not test. A heading escape triggered only by speed reductions did not reduce self-stops in our evaluation. The correction penalty [10] is an established method. In their formulation the certified action respects the input constraints. In our implementation, u^{exec} is the action after the action bounds and the filter. The term therefore penalises both the filter's truncation and the box clipping of $v \leq 0$. It may therefore act on both mechanisms. In E1a it did act on both, but in opposite directions: fewer self-stops and more filter-blocked stalls, with no net reduction in deadlock. This suggests that a remedy for the learned mechanism should be combined with one for the geometric mechanism, for example a heading-level or planning-level escape. The DWA results point the same way. A controller that keeps its own margin above the filter's is never blocked by the filter, but it then needs a way out of its own local minima.

8 Limitations and threats to validity

- **External validity.** 2D simulation with exact Euler kinematics, static circular obstacles, exact state for the filter and a known map. Nothing was tested on a robot, and no finite simulation establishes real-world safety. The filter's guarantee holds at the sampled states. Between samples, the chord bound limits the penetration to about 1.4 mm for the generated obstacle sizes; continuous-time motion is not covered.

- **Scenario design.** On the original generator, goal clipping shortens many goal distances. With honest distances (E3), expert success roughly halves, so success rates on the original sets should not be compared with other benchmarks. Three trivial scenarios add 1–2 points to every success rate in their stages. The layouts are fairly open: the median oracle path is 6–8 % longer than the straight line. No feasibility oracle was run on the E3 scenarios.

- **Test-set reuse.** The margin, the sampled-action evaluation and the stall analysis were all developed on the same test scenarios. The pre-registered evaluation on 500 unused scenarios reduces, but does not remove, this threat. Those scenarios come from the same generator, and only the expert stage, mean actions and one value of κ were tested. The sampled-action variant of that evaluation and the final checkpoints of the agents trained without the filter were planned but not run.

- **Statistical power.** Five seeds per variant, with 100 scenarios per stage on the original test set, 500 on the fresh sets and 1,000 per stage on the corrected generator. With five seeds, the seed-level spread is large (for example, the E1a penalty arm's filter-blocked rate ranged from 6 % to 40 % across seeds), so effects of a few points cannot be resolved. The intervals are approximate, and there is no multiplicity correction beyond the pre-registered decision rules.

- **Artefacts.** The final checkpoints used for the checkpoint-selection analysis are not yet in the public repository.
- **Construct validity of the taxonomy.** The class thresholds are heuristic, but varying the displacement threshold, the share threshold and the window length changed the expert counts by at most 4.
- **Baselines and tuning.** PPO hyperparameters were not tuned for the filtered setting, and the potential field was tuned by hand. The DWA-style planner was tuned only on 12 configurations; its filter-aware variant was exploratory and chosen after seeing results. PPO-Lagrangian and MPC with a DT-CBF are missing. The correction-penalty arms were trained for 300k steps, not 10^6, and only $\alpha = 1$ was tested.
- **Privileged feature.** The proximity feature in the observation is computed from the true map.

9 Conclusion

A one-step discrete-time CBF filter makes learned navigation policies collision-free in simulation, but it does not make them live. Two elementary properties of the filter frame the analysis: its one-step guarantee is independent of the turn rate, and the headings admitting motion collapse to a half-plane at the boundary. Empirically, learned policies deadlock behind the filter by two distinct mechanisms. A policy trained without the filter is blocked by the filter at the boundary, while a policy trained with the filter mostly stops itself before it. Stochastic deployment removes most of the second mechanism but not the first, and with longer goal distances it adds some of the first. A test-time heading escape reduces the first modestly and leaves the second untouched. In these runs, longer training with the filter made the second more frequent. Three training interventions removed most or all self-stops: the published correction penalty (partly), a Beta policy and a reward without the risk term. None of them removed the deadlock. The stalls moved to the filter boundary instead, and total deadlock changed by at most about 5 points. A classical local planner showed the same exchange in reverse. In these experiments, the two mechanisms therefore behave as two outcomes of one underlying problem: the policies have no manoeuvre that makes progress around an obstacle between them and the goal. Remedies that only remove one outcome, whether in training or at test time, do not resolve it. With honest goal distances, success is much lower than on the original generator. These results come from five seeds per arm, 300k-step training for the new arms, and simulation only.

Data and code availability

Code, committed checkpoints, per-episode results and analysis scripts: `https://github.com/pouyapd/SafeNav-RL`. Benchmark: `results/benchmark/`; sampled-action evaluation: `results/stochastic_eval/`; stall taxonomy: `research/scripts/stall_taxonomy.py`; figures and statistics in this paper: `research/scripts/paper_figures_stats.py`; pre-registered fresh-scenario evaluation, heading escape and checkpoint comparison: `research/experiments/lightweight/` (protocol, runner, analysis and per-episode results); correction-penalty experiment E1a (protocol, run records, checkpoints, per-episode results): `research/experiments/E1_correction_penalty.md` and `research/experiments/e1a/`; corrected generator, self-stop experiment and DWA-style planner (protocol, run records, checkpoints, per-episode results, analysis): `research/experiments/round2/`. [**EVIDENCE REQUIRED:** Archive a tagged release with a DOI (e.g. Zenodo) before submission.]

References

[1] Kai-Chieh Hsu, Haimin Hu, and Jaime F. Fisac. The safety filter: A unified view of safety-critical control in autonomous systems. *Annual Review of Control, Robotics, and Autonomous Systems*, 7(1):47–72, 2024. doi: 10.1146/annurev-control-071723-102940.

[2] Lukas Brunke, Melissa Greeff, Adam W. Hall, Zhaocong Yuan, Siqi Zhou, Jacopo Panerati, and Angela P. Schoellig. Safe learning in robotics: From learning-based control to safe reinforcement learning. *Annual Review of Control, Robotics, and Autonomous Systems*, 5(1):411–444, 2022. doi: 10.1146/annurev-control-042920-020211.

[3] Aaron D. Ames, Xiangru Xu, Jessy W. Grizzle, and Paulo Tabuada. Control barrier function based quadratic programs for safety critical systems. *IEEE Transactions on Automatic Control*, 62(8):3861–3876, 2017. doi: 10.1109/TAC.2016.2638961.

[4] Aaron D. Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. Control barrier functions: Theory and applications. In *2019 18th European Control Conference (ECC)*, pages 3420–3431, 2019. doi: 10.23919/ECC.2019.8796030.

[5] Matheus F. Reis, A. Pedro Aguiar, and Paulo Tabuada. Control barrier function-based quadratic programs introduce undesirable asymptotically stable equilibria. *IEEE Control Systems Letters*, 5(2):731–736, 2021. doi: 10.1109/LCSYS.2020.3004797.

[6] Xiao Tan and Dimos V. Dimarogonas. On the undesired equilibria induced by control barrier function based quadratic programs. *Automatica*, 159:111359, 2024. doi: 10.1016/j.automatica.2023.111359.

[7] Jaskaran Grover, Changliu Liu, and Katia Sycara. The before, during, and after of multi-robot deadlock. *The International Journal of Robotics Research*, 42(6):317–336, 2023. doi: 10.1177/02783649221074718.

[8] Mrdjan Jankovic, Mario Santillo, and Yan Wang. Multiagent systems with CBF-based controllers: Collision avoidance and liveness from instability. *IEEE Transactions on Control Systems Technology*, 32(2):705–712, 2024. doi: 10.1109/TCST.2023.3324531.

[9] Rohan Chandra, Vrushabh Zinage, Efstathios Bakolas, Peter Stone, and Joydeep Biswas. Deadlock-free, safe, and decentralized multi-robot navigation in social mini-games via discrete-time control barrier functions. *Autonomous Robots*, 49(2), 2025. doi: 10.1007/s10514-025-10194-8.

[10] Federico Pizarro Bejarano, Lukas Brunke, and Angela P. Schoellig. Safety filtering while training: Improving the performance and sample efficiency of reinforcement learning agents. *IEEE Robotics and Automation Letters*, 10(1):788–795, 2025. doi: 10.1109/LRA.2024.3512374.

[11] Lizhi Yang, Blake Werner, Massimiliano de Sa, and Aaron D. Ames. CBF-RL: Safety filtering reinforcement learning in training with control barrier functions. In *2026 IEEE International Conference on Robotics and Automation (ICRA)*, pages 17825–17832, 2026. doi: 10.1109/ICRA57385.2026.11695808.

[12] Hanna Krasowski, Jakob Thumm, Marlon Müller, Lukas Schäfer, Xiao Wang, and Matthias Althoff. Provably safe reinforcement learning: Conceptual analysis, survey, and benchmarking. *Transactions on Machine Learning Research*, 2023. No DOI; arXiv:2205.06750.

[13] Donggeon David Oh, Duy P. Nguyen, Haimin Hu, and Jaime Fernández Fisac. Provably optimal reinforcement learning under safety filtering. *Proceedings of IASEAI Conference*, 2(1):488–500, 2026. doi: 10.1609/iaseai.v2i1.43046.

[14] Jakob Thumm, Guillaume Pelat, and Matthias Althoff. Reducing safety interventions in provably safe reinforcement learning. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7515–7522, 2023. doi: 10.1109/IROS55552.2023.10342464.

[15] Arko Banerjee, Kia Rahmani, Joydeep Biswas, and Isil Dillig. Dynamic model predictive shielding for provably safe reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS 2024)*, 2024. arXiv:2405.13863.

[16] Peter Trautman and Andreas Krause. Unfreezing the robot: Navigation in dense, interacting crowds. In *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 797–803, 2010. doi: 10.1109/IROS.2010.5654369.

[17] Ayush Agrawal and Koushil Sreenath. Discrete control barrier functions for safety-critical control of discrete systems with application to bipedal robot navigation. In *Robotics: Science and Systems XIII*, 2017. doi: 10.15607/RSS.2017.XIII.073.

[18] Mohammed Alshiekh, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum, and Ufuk Topcu. Safe reinforcement learning via shielding. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 2018. doi: 10.1609/aaai.v32i1.11797.

[19] William Saunders, Girish Sastry, Andreas Stuhlmüller, and Owain Evans. Trial without error: Towards safe reinforcement learning via human intervention. In *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, pages 2067–2069, 2018. doi: 10.65109/mzmg4429.

[20] Jun Zeng, Bike Zhang, and Koushil Sreenath. Safety-critical model predictive control with discrete-time control barrier function. In *2021 American Control Conference (ACC)*, pages 3882–3889, 2021. doi: 10.23919/ACC50511.2021.9483029.

[21] Wei Xiao and Calin Belta. High-order control barrier functions. *IEEE Transactions on Automatic Control*, 67(7):3655–3662, 2022. doi: 10.1109/TAC.2021.3105491.

[22] Kim Peter Wabersich and Melanie N. Zeilinger. A predictive safety filter for learning-based control of constrained nonlinear dynamical systems. *Automatica*, 129:109597, 2021. doi: 10.1016/j.automatica.2021.109597.

[23] Shishir Kolathaya and Aaron D. Ames. Input-to-state safety with control barrier functions. *IEEE Control Systems Letters*, 3(1):108–113, 2019. doi: 10.1109/LCSYS.2018.2853698.

[24] Ryan K. Cosner, Andrew W. Singletary, Andrew J. Taylor, Tamas G. Molnar, Katherine L. Bouman, and Aaron D. Ames. Measurement-robust control barrier functions: Certainty in safety with uncertainty in state. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 6286–6291, 2021. doi: 10.1109/IROS51168.2021.9636584.

[25] Ryan Cosner, Preston Culbertson, Andrew Taylor, and Aaron Ames. Robust safety under stochastic uncertainty with discrete-time control barrier functions. In *Robotics: Science and Systems XIX*, 2023. doi: 10.15607/RSS.2023.XIX.084.

[26] Ersin Daş, Rahal Nanayakkara, Xiao Tan, Ryan M. Bena, Joel W. Burdick, Paulo Tabuada, and Aaron D. Ames. Safe navigation under state uncertainty: Online adaptation for robust control barrier functions. *IEEE Robotics and Automation Letters*, 11(3):3398–3405, 2026. doi: 10.1109/LRA.2026.3653366.

[27] Jaskaran Singh Grover, Changliu Liu, and Katia Sycara. Deadlock analysis and resolution for multi-robot systems. In *Springer Proceedings in Advanced Robotics*, pages 294–312. Springer, 2021. doi: 10.1007/978-3-030-66723-8_18.

[28] Y. Koren and J. Borenstein. Potential field methods and their inherent limitations for mobile robot navigation. In *Proceedings. 1991 IEEE International Conference on Robotics and Automation*, pages 1398–1404, 1991. doi: 10.1109/ROBOT.1991.131810.

[29] Kai-Chieh Hsu, Vicenç Rubies-Royo, Claire Tomlin, and Jaime F. Fisac. Safety and liveness guarantees through reach-avoid reinforcement learning. In *Robotics: Science and Systems XVII*, 2021. doi: 10.15607/RSS.2021.XVII.077.

[30] Steven Carr, Nils Jansen, Sebastian Junges, and Ufuk Topcu. Safe reinforcement learning via shielding under partial observability. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(12):14748–14756, 2023. doi: 10.1609/aaai.v37i12.26723.

[31] Richard Cheng, Gábor Orosz, Richard M. Murray, and Joel W. Burdick. End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(01):3387–3395, 2019. doi: 10.1609/aaai.v33i01.33013387.

[32] Yousef Emam, Gennaro Notomista, Paul Glotfelter, Zsolt Kira, and Magnus Egerstedt. Safe reinforcement learning using robust control barrier functions. *IEEE Robotics and Automation Letters*, 10(3):2886–2893, 2025. doi: 10.1109/LRA.2022.3216996.

[33] Wei Xiao, Tsun-Hsuan Wang, Ramin Hasani, Makram Chahine, Alexander Amini, Xiao Li, and Daniela Rus. BarrierNet: Differentiable control barrier functions for learning of safe robot control. *IEEE Transactions on Robotics*, 39(3):2289–2307, 2023. doi: 10.1109/TRO.2023.3249564.

[34] Tu-Hoa Pham, Giovanni De Magistris, and Ryuki Tachibana. OptLayer - practical constrained optimization for deep reinforcement learning in the real world. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6236–6243, 2018. doi: 10.1109/ICRA.2018.8460547.

[35] Lixing Song, Luke Ferderer, and Shaoen Wu. Safe reinforcement learning for LiDAR-based navigation via control barrier function. In *2022 21st IEEE International Conference on Machine Learning and Applications (ICMLA)*, pages 264–269, 2022. doi: 10.1109/ICMLA55696.2022.00045.

[36] Luca Marzari, Francesco Trotti, Enrico Marchesini, and Alessandro Farinelli. Designing control barrier function via probabilistic enumeration for safe reinforcement learning navigation. *IEEE Robotics and Automation Letters*, 10(10):9630–9637, 2025. doi: 10.1109/LRA.2025.3596431.

[37] Junjun Xie, Shuhao Zhao, Liang Hu, and Huijun Gao. Certificated actor-critic: Hierarchical reinforcement learning with control barrier functions for safe navigation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5482–5488, 2025. doi: 10.1109/ICRA55743.2025.11127225.

[38] Bingbing Guo, Guina Wang, Yiyang Chen, Yue Gao, and Qian Xie. Risk-aware reinforcement learning with dynamic safety filter for collision risk mitigation in mobile robot navigation. *Sensors*, 25(17):5488, 2025. doi: 10.3390/s25175488.

[39] Shaghayegh Keyumarsi, Made Widhi Surya Atman, and Azwirman Gusrialdi. LiDAR-based online control barrier function synthesis for safe navigation in unknown environments. *IEEE Robotics and Automation Letters*, 9(2):1043–1050, 2024. doi: 10.1109/LRA.2023.3339059.

[40] Kehan Long, Cheng Qian, Jorge Cortes, and Nikolay Atanasov. Learning barrier functions with memory for robust safe navigation. *IEEE Robotics and Automation Letters*, 6(3):4931–4938, 2021. doi: 10.1109/LRA.2021.3070250.

[41] Yasuhiro Fujita and Shin-ichi Maeda. Clipped action policy gradient. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1597–1606, 2018. URL `https://proceedings.mlr.press/v80/fujita18a.html`.

[42] Po-Wei Chou, Daniel Maturana, and Sebastian Scherer. Improving stochastic policy gradients in continuous control with deep reinforcement learning using the Beta distribution. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 834–843, 2017. URL `https://proceedings.mlr.press/v70/chou17a.html`.

[43] Jyun-Li Lin, Wei Hung, Shang-Hsuan Yang, Ping-Chun Hsieh, and Xi Liu. Escaping from zero gradient: Revisiting action-constrained reinforcement learning via Frank-Wolfe policy optimization. In *Proceedings of the Thirty-Seventh Conference on Uncertainty in Artificial Intelligence*, volume 161 of *Proceedings of Machine Learning Research*, pages 397–407, 2021. URL `https://proceedings.mlr.press/v161/lin21b.html`.

[44] Sebastien Gros, Mario Zanon, and Alberto Bemporad. Safe reinforcement learning via projection on a safe set: How to achieve optimality? *IFAC-PapersOnLine*, 53(2):8076–8081, 2020. doi: 10.1016/j.ifacol.2020.12.2276.

[45] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 2018. doi: 10.1609/aaai.v32i1.11694.

[46] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. No DOI; arXiv:2108.13264.

[47] Daniel Perille, Abigail Truong, Xuesu Xiao, and Peter Stone. Benchmarking metric ground navigation. In *2020 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, pages 116–121, 2020. doi: 10.1109/SSRR50563.2020.9292572.

[48] Linh Kästner, Reyk Carstens, Huajian Zeng, Jacek Kmiecik, Teham Bhuiyan, Niloufar Khorsandhi, Volodymyir Shcherbyna, and Jens Lambrecht. Arena-Rosnav 2.0: A development and benchmarking platform for robot navigation in highly dynamic environments. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 11257–11264, 2023. doi: 10.1109/IROS55552.2023.10342152.

[49] Anthony Francis, Claudia Pérez-D'Arpino, Chengshu Li, Fei Xia, Alexandre Alahi, Rachid Alami, Aniket Bera, Abhijat Biswas, Joydeep Biswas, Rohan Chandra, Hao-Tien Lewis Chiang, Michael Everett, Sehoon Ha, Justin Hart, Jonathan P. How, Haresh Karnan, Tsang-Wei Edward Lee, Luis J. Manso, Reuth Mirsky, Sören Pirk, Phani Teja Singamaneni, Peter Stone, Ada V. Taylor, Peter Trautman, Nathan Tsoi, Marynel Vázquez, Xuesu Xiao, Peng Xu, Naoki Yokoyama, Alexander Toshev, and Roberto Martín-Martín. Principles and guidelines for evaluating social robot navigation algorithms. *ACM Transactions on Human-Robot Interaction*, 14(2):1–65, 2025. doi: 10.1145/3700599.

[50] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. Preprint, arXiv:1707.06347.

[51] D. Fox, W. Burgard, and S. Thrun. The dynamic window approach to collision avoidance. *IEEE Robotics & Automation Magazine*, 4(1):23–33, 1997. doi: 10.1109/100.580977.

[52] Adam Stooke, Joshua Achiam, and Pieter Abbeel. Responsive safety in reinforcement learning by PID Lagrangian methods. In *International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, 2020. No DOI.